

flat_meow Fixes

Document #: P3567R1 [[Latest](#)] [[Status](#)]
Date: 2025-09-06
Project: Programming Language C++
Audience: LEWG, LWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>
Louis Dionne
<ldionne.2@gmail.com>
Arthur O'Dwyer
<arthur.j.odwyer@gmail.com>

Contents

- [1 Revision History](#)
 - [1.1 R1](#)
- [2 Abstract](#)
- [3 Introduction](#)
- [4 LWG 4000: `flat\_map::insert\_range` Fix](#)
 - [4.1 Wording](#)
- [5 `swap` Should be Conditionally `noexcept`](#)
 - [5.1 Wording](#)
- [6 Missing `insert\_range\(sorted\_unique, rg\)`](#)
 - [6.1 Wording](#)
 - [6.1.1 `flat\_map`](#)
 - [6.1.2 `flat\_multimap`](#)
 - [6.1.3 `flat\_set`](#)
 - [6.1.4 `flat\_multiset`](#)
- [7 `flat\_set::insert\_range`](#)
 - [7.1 Wording](#)
- [8 Underspecified special member functions](#)
 - [8.1 Wording](#)
 - [8.1.1 `flat\_map` Wording](#)

- [8.1.2 flat_multimap Wording](#)
- [8.1.3 flat_set Wording](#)
- [8.1.4 flat_multiset Wording](#)
- [9 Implementation Experience](#)
- [10 Feature Test Macro](#)
- [11 References](#)

§ 1 Revision History

§ 1.1 R1

- Wording Fixes

§ 2 Abstract

This paper proposes a subset of the fixes in [\[P2767R2\]](#) to `flat_map`, `flat_multimap`, `flat_set`, and `flat_multiset` based on `libc++`'s implementation.

§ 3 Introduction

[\[P2767R2\]](#) proposed a set of changes to `flat_map`, `flat_multimap`, `flat_set`, and `flat_multiset`, based on the author's implementation experience. However, the paper contains not only some straight forward fixes, but also some design changes, which did not get consensus in previous meetings. We (`libc++`) have recently released the `flat_map` implementation. After consulting the author of the original paper [\[P2767R2\]](#), we decided to create a new paper which contains a subset of the non-controversial fixes based on our implementation experience.

§ 4 LWG 4000: `flat_map::insert_range` Fix

As stated in [\[LWG4000\]](#), `flat_map::insert_range` has an obvious bug. But for some reason, this LWG issue was not moved forward, possibly due to the existence of [\[P2767R2\]](#). The fix is twofold: first, we use `value_type` explicitly to make sure that `e` is a `std::pair`, and we move to `ranges::for_each` for consistency with the rest of the `flat_map` specification.

§ 4.1 Wording

Change 23.6.8.7 [\[flat.map.modifiers\]](#) paragraph 12 to:

```
template<container-compatible-range<value_type> R>
void insert_range(R&& rg);
```

12 *Effects*: Adds elements to `c` as if by:

```
for (const auto& e : rg) {
ranges::for_each(rg, [&](value_type e) {
    c.keys.insert(c.keys.end(), std::move(e.first));
    c.values.insert(c.values.end(), std::move(e.second));
});
```

Then, sorts the range of newly inserted elements with respect to `value_comp()`; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases the duplicate elements as if by:

```
auto zv = views::zip(c.keys, c.values);
auto it = ranges::unique(zv, key_equiv(compare)).begin();
auto dist = distance(zv.begin(), it);
c.keys.erase(c.keys.begin() + dist, c.keys.end());
c.values.erase(c.values.begin() + dist, c.values.end());
```

§ 5 swap Should be Conditionally **noexcept**

`flat_meow::swap` is currently specified as unconditionally **noexcept**. In case the underlying container's `swap` throws an exception, implementations have to either

1. Swallow the exception and restore invariants. Practically speaking, this means clearing both containers.
2. `std::terminate`

The first option is extremely bad because users will silently get an empty `flat_meow` after a failed `swap`. The second option is extremely harsh.

Instead, making `swap` conditionally-**noexcept** allows us to propagate the exception (after restoring invariants).

§ 5.1 Wording

Change 23.6.8.2 [`flat_map.defn`] as follows:

```
void swap(flat_map& y) noexcept(see below);

// ...

friend void swap(flat_map& x, flat_map& y) noexcept(noexcept(x.swap(y)))
{ x.swap(y); }
```

Change 23.6.8.7 [flat.map.modifiers] paragraph 33:

```
void swap(flat_map& y) noexcept;  
    noexcept(is_nothrow_swappable v<key_container_type> &&  
              is_nothrow_swappable v<mapped_container_type> &&  
              is_nothrow_swappable v<key_compare>);
```

33 *Effects*: Equivalent to:

```
ranges::swap(compare, y.compare);  
ranges::swap(c.keys, y.c.keys);  
ranges::swap(c.values, y.c.values);
```

Change 23.6.9.2 [flat.multimap.defn] as follows:

```
void swap(flat_multimap& y) noexcept;  
    noexcept(is_nothrow_swappable v<key_container_type> &&  
              is_nothrow_swappable v<mapped_container_type> &&  
              is_nothrow_swappable v<key_compare>);  
  
// ...  
  
friend      void      swap(flat_multimap&      x,      flat_multimap&      y)  
    noexcept(noexcept(x.swap(y)))  
    { x.swap(y); }
```

Change 23.6.11.2 [flat.set.defn] as follows:

```
void swap(flat_set& y) noexcept(see below);  
  
// ...  
  
friend void swap(flat_set& x, flat_set& y) noexcept(noexcept(x.swap(y)))  
    { x.swap(y); }
```

Change 23.6.11.5 [flat.set.modifiers] paragraph 13:

```
void swap(flat_set& y) noexcept;  
    noexcept(is_nothrow_swappable v<container_type> &&  
              is_nothrow_swappable v<key_compare>);
```

13 *Effects*: Equivalent to:

```
ranges::swap(compare, y.compare);  
ranges::swap(c, y.c);
```

Change 23.6.12.2 [flat.multiset.defn] as follows:

```
void swap(flat_multiset& y) noexcept(see below);  
  
// ...  
  
friend      void      swap(flat_multiset&      x,      flat_multiset&      y)  
    noexcept(noexcept(x.swap(y)))  
    { x.swap(y); }
```

Change 23.6.12.5 [\[flat.multiset.modifiers\]](#) paragraph 9:

```
void swap(flat_multiset& y) noexcept;  
    noexcept(is_nothrow_swappable v<container_type> &&  
        is_nothrow_swappable v<key_compare>);
```

9 *Effects*: Equivalent to:

```
ranges::swap(compare, y.compare);  
ranges::swap(c, y.c);
```

§ 6 Missing `insert_range(sorted_unique, rg)`

The multi-element insertion API consists of two sets of overloads:

- A set of overloads that take a potentially-unsorted range
- A set of overloads that take a `sorted_unique` range

```
insert(first, last);           // 1a  
insert(il);                   // 2a  
insert_range(rg);             // 3a  
  
insert(sorted_unique, first, last); // 1b  
insert(sorted_unique, il);       // 2b  
insert_range(sorted_unique, rg)  // 3b is conspicuously missing.
```

However, the last overload `insert_range(sorted_unique, rg)` is missing. This could be an oversight in the original proposal. This is true for `flat_set` and `flat_map`. Similarly, in `flat_multiset` and `flat_multimap`, the overload `insert_range(sorted_equivalent, rg)` is also missing.

§ 6.1 Wording

§ 6.1.1 `flat_map`

Change 23.6.8.2 [\[flat.map.defn\]](#) as follows:

```
template<class InputIterator>  
    void insert(InputIterator first, InputIterator last);  
template<class InputIterator>  
    void insert(sorted_unique_t, InputIterator first, InputIterator last);  
template<container-compatible-range<value_type> R>  
    void insert_range(R&& rg);  
template<container-compatible-range<value_type> R>  
void insert_range(sorted_unique_t, R&& rg);
```

Add a new entry to 23.6.8.7 [\[flat.map.modifiers\]](#) after paragraph 14:

```
template<container-compatible-range<value_type> R>  
    void insert_range(sorted_unique_t, R&& rg);
```

15 *Effects*: Equivalent to `insert_range(rg)`.

16 *Complexity*: Linear in N , where N is `size()` after the operation.

§ 6.1.2 `flat_multimap`

Change 23.6.9.2 [[flat.multimap.defn](#)] as follows:

```
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<class InputIterator>
    void insert(sorted_equivalent_t, InputIterator first, InputIterator
        last);
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
    void insert_range(sorted_equivalent t, R&& rg);
```

§ 6.1.3 `flat_set`

Change 23.6.11.2 [[flat.set.defn](#)] as follows:

```
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<class InputIterator>
    void insert(sorted_unique_t, InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
    void insert_range(sorted_unique t, R&& rg);
```

Add a new entry to 23.6.11.5 [[flat.set.modifiers](#)] after paragraph 12:

```
template<container-compatible-range<value_type> R>
    void insert_range(sorted_unique_t, R&& rg);
```

13 *Effects*: Equivalent to `insert_range(rg)`.

14 *Complexity*: Linear in N , where N is `size()` after the operation.

§ 6.1.4 `flat_multiset`

Change 23.6.12.2 [[flat.multiset.defn](#)] as follows:

```
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<class InputIterator>
    void insert(sorted_equivalent_t, InputIterator first, InputIterator
        last);
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
    void insert_range(sorted_equivalent t, R&& rg);
```

Change 23.6.12.5 [flat.multiset.modifiers] Paragraph 8 as follows:

```
template<class InputIterator>
    constexpr void insert(sorted_equivalent_t, InputIterator first,
                          InputIterator last);
```

7 *Effects*: Equivalent to `insert(first, last)`.

8 *Complexity*: Linear in N , where N is `size()` after the operation.

Add a new entry to 23.6.12.5 [flat.multiset.modifiers] after Paragraph 8:

```
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
```

9 *Effects*: Adds elements to `c` as if by:

```
ranges::for_each([&](auto&& e) {
    c.insert(c.end(), std::forward<decltype(e)>(e));
});
```

Then, sorts the range of newly inserted elements with respect to `compare`, and merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range.

10 *Complexity*: $N + M \log M$, where N is `size()` before the operation and M is `ranges::distance(rg)`.

11 *Remarks*: Since this operation performs an in-place merge, it may allocate memory.

```
template<container-compatible-range<value_type> R>
    void insert_range(sorted_equivalent_t, R&& rg);
```

12 *Effects*: Equivalent to `insert_range(rg)`.

13 *Complexity*: Linear in N , where N is `size()` after the operation.

§ 7 flat_set::insert_range

The current specification for `flat_set::insert_range` seems to unnecessarily pessimize by forcing copies of the elements:

```
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
```

10 *Effects*: Adds elements to `c` as if by:

```
for (const auto& e : rg) {
    c.insert(c.end(), e); // COPYING HERE
}
```

We should allow implementations to move when they can.

§ 7.1 Wording

Change 23.6.11.5 [flat.set.modifiers] paragraph 10 as follows:

```
template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
```

10 *Effects*: Adds elements to *c* as if by:

```
for (const auto& e : rg) {
    ranges::for_each(rg, [&](auto&& e) {
        c.insert(c.end(), std::forward<decltype(e)>(e));
    })
```

Then, sorts the range of newly inserted elements with respect to *compare*; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases all but the first element from each group of consecutive equivalent elements.

§ 8 Underspecified special member functions

The special member functions for `flat_meow` are currently not specified explicitly. This means that an implementation using e.g. `flat_map(flat_map&&) = default` would be conforming. However, such an implementation would not be correct with respect to exception handling. Indeed, if an exception is thrown while moving from the incoming map, the incoming map would be left in a potentially invalid state with respect to its invariants.

Note that the [blanket paragraph](#) does not apply here, since we're concerned with the incoming `flat_map`'s invariants, not `*this`'s invariants. We believe that the behavior of these special member functions must be specified explicitly, otherwise these constructors are useless in any context where an exception can be thrown.

§ 8.1 Wording

§ 8.1.1 flat_map Wording

Change 23.6.8.1 [flat.map.overview] Paragraph 6 as follows:

- ⁶ If any member function in 23.6.8.2 [flat.map.defn] exits via an exception, the invariants are restored. For the move constructor and move assignment operator, the invariants of the argument are also restored.

[Note 2: This can result in the `flat_map` being emptied. — *end note*]

Change 23.6.8.2 [flat.map.defn] as follows:


```
// [flat.map.cons], constructors
flat_map() : flat_map(key_compare()) { }

flat_map(const flat_map&);
flat_map(flat_map&&);
flat_map& operator=(const flat_map&);
flat_map& operator=(flat_map&&);
```

§ 8.1.2 flat_multimap Wording

Change 23.6.9.1 [flat.multimap.overview] Paragraph 6 as follows:

- 6 If any member function in 23.6.9.2 [flat.multimap.defn] exits via an exception, the invariants are restored. For the move constructor and move assignment operator, the invariants of the argument are also restored.

[Note 2: This can result in the flat_multimap being emptied. — *end note*]

Change 23.6.9.2 [flat.multimap.defn] as follows:

```
// [flat.multimap.cons], constructors
flat_multimap() : flat_multimap(key_compare()) { }

flat_multimap(const flat_multimap&);
flat_multimap(flat_multimap&&);
flat_multimap& operator=(const flat_multimap&);
flat_multimap& operator=(flat_multimap&&);
```

§ 8.1.3 flat_set Wording

Change 23.6.11.1 [flat.set.overview] Paragraph 6 as follows:

- 6 If any member function in 23.6.11.2 [flat.set.defn] exits via an exception, the invariant is restored. For the move constructor and move assignment operator, the invariant of the argument is also restored.

[Note 2: This can result in the flat_set's being emptied. — *end note*]

Change 23.6.11.2 [flat.set.defn] as follows:

```
// [flat.set.cons], constructors
flat_set() : flat_set(key_compare()) { }

flat_set(const flat_set&);
flat_set(flat_set&&);
flat_set& operator=(const flat_set&);
flat_set& operator=(flat_set&&);
```

§ 8.1.4 flat_multiset Wording

Change 23.6.12.1 [flat.multiset.overview] Paragraph 6 as follows:

- ⁶ If any member function in 23.6.12.2 [flat.multiset.defn] exits via an exception, the invariant is restored. For the move constructor and move assignment operator, the invariant of the argument is also restored.

[Note 2: This can result in the flat_multiset's being emptied. — *end note*]

Change 23.6.12.2 [flat.multiset.defn] as follows:

```
// [flat.multiset.cons], constructors
flat_multiset() : flat_multiset(key_compare()) { }

flat_multiset(const flat_multiset&);
flat_multiset(flat_multiset&&);
flat_multiset& operator=(const flat_multiset&);
flat_multiset& operator=(flat_multiset&&);
```

§ 9 Implementation Experience

This paper is based on our implementation in libc++.

§ 10 Feature Test Macro

Bump __cpp_lib_flat_map and __cpp_lib_flat_set appropriately.

§ 11 References

[LWG4000] Hewill Kang. flat_map::insert_range's Effects is not quite right.

<https://wg21.link/lwg4000>

[P2767R2] Arthur O'Dwyer. 2023-12-09. flat_map/flat_set omnibus.

<https://wg21.link/p2767r2>